

Me: should i create my own atlas cluster easier because my friends are not responding and here is the code for the task routes: `import express from "express"; import Task from "../models/Task.js"; const router = express.Router(); router.get("/", async (req, res) => { try { const tasks = await Task.find().populate("event assignedTo"); res.json(tasks); } catch (err) { res.status(500).json({ message: err.message }); } }); router.post("/", async (req, res) => { try { const task = await Task.create(req.body); res.status(201).json(task); } catch (err) { res.status(400).json({ message: err.message }); } }); router.put("/:id", async (req, res) => { try { const task = await Task.findByIdAndUpdate(req.params.id, req.body, { new: true }); if (!task) return res.status(404).json({ message: "Task not found" }); res.json(task); } catch (err) { res.status(400).json({ message: err.message }); } }); export default router;`

AI: Yes. Given that:

- Your teammates aren't responding.
- The current Atlas cluster is failing.
- You're Member 2 (backend lead).
- The project deadline matters more than preserving Ziad's old database.

I would **create your own MongoDB Atlas cluster today**.

Also, your `task.routes.js` looks correct:

```
import Task from "../models/Task.js";
```

✅ Correct path.

So I'm not seeing route import problems there.

What I recommend

Step 1: Create Atlas account

Go to:

[MongoDB Atlas](#)

Create a free account.

Step 2: Create a free cluster

Choose:

- FREE tier (M0)
- AWS

- Any region near Germany

Wait until it finishes creating.

Step 3: Create database user

Atlas → Database Access

Create:

Username: member2

Password: (generate one)

Save the password somewhere.

Step 4: Allow your IP

Atlas → Network Access

Add:

0.0.0.0/0

Choose:

Allow access from anywhere

For a university project this is fine.

Step 5: Get connection string

Click:

Connect

→ Drivers

→ Node.js

Atlas will show something like:

<mongodb+srv://member2:<password>@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority>

Replace:

<password>

with your actual password.

Then add your database name:

/event_platform

before the ?.

Example:

[mongodb+srv://member2:MyPassword123@cluster0.xxxxx.mongodb.net/event_platform?
retryWrites=true&w=majority](mongodb+srv://member2:MyPassword123@cluster0.xxxxx.mongodb.net/event_platform?retryWrites=true&w=majority)

Step 6: Update .env

Replace:

MONGO_URI=old_uri_here

with your new Atlas URI.

Save the file.

Step 7: Restart backend

In terminal:

```
npm run dev
```

What I expect to happen

Instead of:

MongoDB connection error

you should see:

MongoDB connected

Server running on port 5001

If that happens:

✅ Database fixed

✅ Backend running

✅ You can test APIs

✅ Member 2 work becomes demonstrable

When you reach the screen where Atlas shows the connection string, send me a screenshot and I'll verify the URI before you paste it into `.env`.

Me : What should I do now and should I download the env file

Ai: Perfect. You're on the right screen.

First: DO NOT download the `.env` file

You don't need it.

Second: VERY IMPORTANT

The screenshot contains your new MongoDB username and password.

After we're done, I recommend creating a new password because the credentials are now exposed. For now, continue.

What to do next

Click the green button:

Choose a connection method

Then:

1. Select **Drivers**
2. Select **Node.js** if it asks
3. Atlas will show you a connection string that looks like:

mongodb+srv://username:<db_password>@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority

Send me a screenshot of THAT screen

Don't paste the password manually. Just show the connection string screen and I'll tell you exactly what to copy into your existing `.env`.

One more thing

Before we test the connection later, go to:

Network Access → Add IP Address

and make sure you have either:

`0.0.0.0/0`

(Allow access from anywhere)

or your current IP address is listed.

For a university project, `0.0.0.0/0` is easiest.

Me: I replaced the uri and this is the output

Ai: Good news: the error in the first screenshot is actually a different mistake.

You are now here:

```
C:\Users\Karee\Documents\GitHub\SE_MILESTONE_AYAZK>
```

But your `package.json` is inside:

```
SE_MILESTONE_AYAZK\backend
```

So when you ran:

```
npm run dev
```

Windows said:

```
ENOENT: no such file or directory, open package.json
```

because you're in the wrong folder.

[Do this exactly](#)

In the terminal:

```
cd backend
```

Then verify:

```
pwd
```

You should see something ending with:

```
...\SE_MILESTONE_AYAZK\backend
```

Then run:

```
npm run dev
```

One more thing I spotted

Your .env currently contains:

[MONGO_URI=mongodb+srv://kareemelhamahmy03_db_user:JKQzOhR0bAChR4Rg@cluster0.fnrkts.mongodb.net/event_platform?retryWrites=true&w=majority&appName=Cluster0](mongodb+srv://kareemelhamahmy03_db_user:JKQzOhR0bAChR4Rg@cluster0.fnrkts.mongodb.net/event_platform?retryWrites=true&w=majority&appName=Cluster0)

This looks fine.

Next step

Run:

```
cd backend
npm run dev
```

Me: so i paste the 2 lines in the same terminal run

Al: Yes. Use the **same terminal** and run them one after the other:

```
$env:NODE_OPTIONS="--dns-result-order=ipv4first"
npm run dev
```

It should look like:

```
PS C:\Users\Karee\Documents\GitHub\SE_MILESTONE_AYAZK\backend>
$env:NODE_OPTIONS="--dns-result-order=ipv4first"
```

```
PS C:\Users\Karee\Documents\GitHub\SE_MILESTONE_AYAZK\backend> npm run dev
```

Don't put them on one line.

After `npm run dev`, send me a screenshot of the output. The important thing is whether:

MongoDB connected

appears,

or if it still says:

querySrv ECONNREFUSED

Send the result and we'll know the next step immediately.

Me: It says login failed

Ai: Perfect. This confirms the frontend is trying to call the backend.

The reason for "**Login failed**" is almost certainly:

Frontend → Backend API → MongoDB



Connection failed

Since your backend crashes immediately with:

MongoDB connection error:

querySrv ECONNREFUSED

the login API cannot work.

[Fastest thing to do now](#)

Click **F12** in Chrome.

Then:

1. Click **Network**
2. Click **Login** again
3. Look for a red request

OR

Open a new terminal in the frontend folder and run:

`type package.json`